NFC Smart Lock

Firmware Engineering Specification

Session, Application, Communication Facade, Transport, and Hardware/Link Layer (ESP32)

Version 0.1 – Draft (Pre-Release)

August 4, 2026

Abstract

This document is the engineering specification for the Lock-side (ESP32) firmware of the NFC Smart Lock system. It defines, as normative requirements, what each firmware component must do at its boundaries – message formats, state machines, interface contracts, timing guarantees, and security properties – without describing how any component is implemented internally. Implementation detail (struct layouts, driver call sequences, task/ISR wiring, and source-level code) is intentionally excluded from this document and lives instead in the companion implementation references: `session.md`, `transport.md`, `lli.md`, `comm_module.md`, and the `Communication_Module_Master.md` reference that ties them together. Where this document and an implementation reference could be read as disagreeing, this document states the normative requirement; the implementation reference should be brought into conformance with it, or the discrepancy is called out explicitly below as an open item.

This document covers five collaborating components that together make up the firmware: the **Session Module** (cryptographic identity and secure channel), the **Application Module** (command dispatch, authorization, and lock actuation), the **Communication Module Facade** (the single integration boundary the Application Module is permitted to depend on), the **Transport** (ISO-DEP protocol state machine), and the **Hardware/Link Layer** (the concrete PN532 binding). A companion *Mobile Application Engineering Specification*, covering the Flutter/phone side of the protocol, is planned as a separate document and is out of scope here.

Contents

Document Map and Reading Order

Part	Covers
I. Session Module	Ed25519/X25519/HKDF/AES-GCM identity, handshake, peer resolution, secure messaging, session termination
II. Application Module	Plaintext command set, authorization, lock actuation, mechanical error handling, provisioning workflow
III. Communication Module Facade	The lifecycle, command-handoff, and event-notification contract between the Application task and everything beneath it
IV. Transport	ISO/IEC 14443-4 APDU framing, instruction set, protocol state machine, timing
V. Hardware / Link Layer	PN532 binding of Transport's abstract Link Layer Interface (LLI)
VI. Document Set History	Provenance of this document and its relationship to prior specifications

Numbering note: this project has not reached a stable v1.0 release. This document's version (0.1) is independent of, and not comparable to, the version of any prior document it supersedes or any document it will be split from in the future (e.g. a Mobile Application specification).
Layering note: Parts I and II describe the Session and Application Modules as collaborating peers – neither is built on top of the other, and neither owns the other's external dependency (Transport for Session; lock actuator hardware for Application). Part III is the genuine integration point between them: it is a real dependency the Application Module has on the rest of the firmware, exposed through exactly one header. Part IV, in turn, is a genuine dependency of Session's byte transport on a conforming implementation of Part V. See the architecture diagram in Part III, §2.

Part I
Session Module

1 Scope and Design Assumptions

This part specifies cryptographic session establishment and secure messaging between the Phone and the Lock. The following scope decisions are in effect for this revision and are treated as explicit, reviewed trade-offs rather than oversights.

- **Handshake structure:** three-message mutual authentication (SIGMA-style) to eliminate signature-transcript ambiguity. See §4.1.
- **Secure element:** the Lock currently stores its long-term Ed25519 private key in software. Migration to an ATECC608A secure element is planned; see §9.1.
- **Session message volume:** each NFC session carries a low number of application messages. This justifies randomly generated AES-GCM nonces without a counter construction; see §7.2.
- **Relay attacks:** out of scope for this revision; see §9.2.

1.1 Non-Goals

This part does not cover: provisioning workflows in the sense of secret distribution, QR rendering, or persistent identity storage (Part II §7), transport framing or ISO-DEP timing (Part IV), NFC controller specifics (Part V), physical actuation, or anything that happens to plaintext bytes once handed to the Application Module (Part II). This module authenticates devices; it does not authorize actions.

2 System Architecture

2.1 Components

Component	Platform	Role (this module)
Mobile Application	Flutter (iOS/Android)	Initiates the handshake, encrypts/decrypts application payloads. Out of scope; see the forthcoming Mobile Application specification.
Lock Controller	ESP32 (ESP-IDF)	Verifies Phone identity, performs the handshake, encrypts/decrypts application payloads. Does not interpret plaintext content or actuate hardware.

2.2 Cryptographic Primitives

Primitive	Use
Ed25519 (RFC 8032)	Long-term identity signatures over the handshake transcript
X25519 (RFC 7748)	Ephemeral key agreement, all-zero output rejected
HKDF-SHA256 (RFC 5869)	Session-key derivation
AES-256-GCM	Authenticated encryption of application payloads
CSPRNG	Nonces, ephemeral keys, challenges

No custom cryptographic primitives are implemented. The specific library bound to each primitive is an implementation detail (`session.md` §"Crypto Backend") and is not repeated here; this specification is written so that primitive substitution (e.g. a future ATECC608A backend, §9.1) requires no change to this document.

3 Long-Term Identity

Each device holds a permanent Ed25519 identity key pair, generated once at provisioning time (Part II §7).

```
Phone:  Ed25519 Private Key (Keychain / Keystore)
        Ed25519 Public Key  (provisioned to Lock at first pairing)

Lock:   Ed25519 Private Key (software; ATECC608A in a future revision)
        Ed25519 Public Key  (provisioned to Phone during pairing)
```

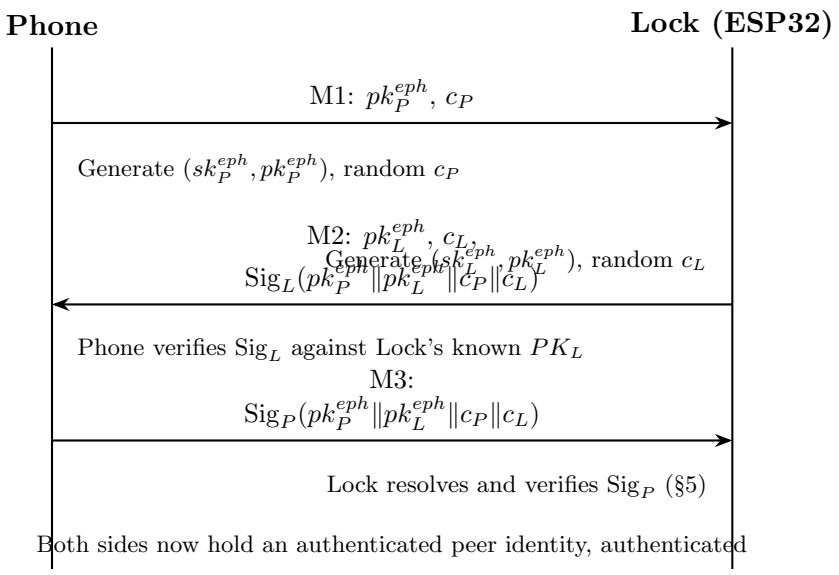
Public key distribution is governed by the provisioning workflow (Part II §7) and is otherwise outside the scope of this Part. This specification assumes both parties hold an authentic copy of the peer's Ed25519 public key before ordinary (non-provisioning) handshakes begin.

4 Handshake Protocol

4.1 Rationale for the Three-Message Design

A two-message design in which the Phone signs only its own ephemeral key and challenge is not bound to the Lock's contribution to the session, permitting a valid Phone signature from one session to be spliced into another session carrying a different Lock challenge. This specification therefore requires a SIGMA-style three-message handshake in which every signature produced by either party covers *both* ephemeral public keys and *both* challenges, so no signature is valid outside the session it was created for.

4.2 Message Flow



4.3 Message Definitions

Msg	Sender	Contents
M1	Phone	pk_P^{eph} (X25519, 32 bytes), c_P (32 bytes). Carries **no identity hint** – see §5.
M2	Lock	pk_L^{eph} (32 bytes), c_L (32 bytes), $\mathrm{Sig}_L(pk_P^{eph} \| pk_L^{eph} \| c_P \| c_L)$ (Ed25519, 64 bytes)
M3	Phone	$\mathrm{Sig}_P(pk_P^{eph} \| pk_L^{eph} \| c_P \| c_L)$ (64 bytes)

The signed transcript SHALL be domain-separated: `"SLOCK-HS-v1"` $\|$ a one-byte protocol version $\|$ $pk_P^{eph} \| pk_L^{eph} \| c_P \| c_L$ (140 bytes total), identical on both platforms, to prevent cross-protocol signature reuse.

4.4 Verification Rules

1. Upon receiving M2, the Phone MUST verify Sig_L against PK_L before proceeding to M3; on failure it MUST abort without deriving any session key material.

2. Upon receiving M3, the Lock MUST resolve and verify Sig_P (§5) before treating the session as authenticated; on failure it MUST abort and MUST NOT signal a successful handshake to the Application Module.

3. Both parties MUST reject a handshake if pk_P^{eph} or pk_L^{eph} is the all-zero point or otherwise fails X25519 validity checks.

4. Challenges c_P, c_L MUST be generated with a CSPRNG and MUST NOT be reused across sessions.

5 Peer Identity Resolution

M1 carries no identity hint (§4.2), so the Lock cannot look up which provisioned public key to verify M3 against in advance; it can only **enumerate candidates**.

Requirement: the Lock SHALL resolve the peer identity for M3 by iterating a caller-supplied enumerator of candidate long-term Ed25519 public keys, in index order starting at zero, attempting Ed25519 verification of Sig_P against each candidate in turn:

- The first candidate whose verification succeeds is the authenticated identity for the session; the Lock SHALL proceed to key derivation (§6) using that outcome.
- Enumeration SHALL stop, and the handshake SHALL be treated as an authentication failure indistinguishable from a single bad signature, when either the enumerator signals exhaustion or a defensive bound of 64 candidates is reached, whichever comes first. The bound exists only to protect against a misbehaving enumerator; it does not change behavior for a correctly implemented one.
- Any candidate that verifies is, by definition, an authorized long-term identity for the purposes of this module – authentication implies authorization at the Session layer (Part II §5 governs whether that identity may issue a given command).

This is the same trust model as an `authorized_keys` file: per-candidate verification cost is small (single-digit milliseconds), the realistic candidate count is small (an owner plus a handful of guests), and the full linear scan fits comfortably inside the timing budget described in Part IV §8.2 even at several dozen candidates. If the candidate count ever grows large enough for linear scan to become a real cost, replacing what backs the enumerator (e.g. an indexed store) is an Application-layer concern (§9.1, Part II §9.2) and does not change this resolution contract.

6 Shared Secret and Key Derivation

6.1 Shared Secret

Both devices independently compute the X25519 shared secret:

```
SharedSecret = X25519(local_ephemeral_private, remote_ephemeral_public)
```

6.2 Key Derivation

```
PRK  = HKDF-Extract(salt = c_P || c_L, ikm = SharedSecret)
K_p2e = HKDF-Expand(PRK, info = "phone->esp", L = 32)
K_e2p = HKDF-Expand(PRK, info = "esp->phone", L = 32)
```

Two directional 256-bit keys are derived using distinct `info` strings to prevent key reuse across communication directions.

7 Secure Communication

7.1 Message Format

Field	Size	Description
Nonce	12 bytes	CSPRNG-generated, unique per message per key
Ciphertext	variable	AES-256-GCM output for the application-layer plaintext (opaque to this module; see Part II)
Auth Tag	16 bytes	GCM authentication tag

7.2 Plaintext Capacity – Flagged Reconciliation

Two numbers for the maximum application plaintext per unchained message have circulated across this document set, and they are not the same number:

- The original Transport specification computed a **227-byte** plaintext budget by subtracting the 28-byte AES-GCM overhead from the 255-byte maximum Short-APDU `Lc` field (Part IV §4.3).
- The as-built Transport implementation instead enforces `Lc` $\leq$ **227** for the *complete* encrypted package (nonce ‖ ciphertext ‖ tag), not 255. After the 28-byte overhead, this yields a true maximum plaintext of **199 bytes**, not 227.

This specification records the as-built figure (199 bytes) as the currently enforced ceiling, per the implementation-is-source-of-truth principle stated in this document's abstract, and flags the 28-byte gap between the two figures as an open item rather than silently adopting either without comment. None of the commands defined in Part II approach either ceiling today (the largest, `CMD_PROVISION`, is 65 bytes), so this is a latent discrepancy, not a live defect. A future revision MUST do one of: (a) adopt 199 bytes as the frozen budget and correct Part IV §4.3 and this section to match, or (b) relax Transport's enforced threshold to `Lc` $\leq$ 255 to restore the originally intended 227-byte plaintext budget. This document does not decide between them; see Part IV §4.3 for the corresponding Transport-side flag and Part VI for tracking.

Session's internal plaintext buffer capacity MAY be sized to either figure without a security consequence, since the wire-level ceiling described above is enforced independently by Transport regardless of buffer size.

7.3 Nonce Strategy for Low-Volume Sessions

Given the low expected number of messages per session, random 96-bit nonces are acceptable for this revision; the birthday-bound collision risk becomes significant only near 2^{32} messages under one key, and each session uses a fresh key. Should message volume increase substantially, a future revision SHOULD move to a deterministic counter-based nonce.

7.4 Replay Protection Within a Session

Because each session derives a fresh key and is typically short-lived and single-purpose, sequence-number-based replay protection within a session is OPTIONAL in this revision. Whether a given plaintext command is safe to replay (e.g. an idempotent status query vs. a state-changing unlock) is an Application-layer policy decision (Part II).

8 Session Lifecycle

8.1 Stages

The Lock's session state SHALL track exactly which key material currently exists, using three stages: no material exists; ephemeral material exists (post-M1, pre-M3); and full session material exists (post-M3, both directional keys derived). A fresh M1 SHALL always begin by discarding any stale material from a prior, incomplete attempt, so an aborted session can never contaminate the next one.

8.2 Lifecycle Notification

Requirement: the Session Module SHALL expose two lifecycle notification points to whatever component integrates it (Part III):

- A notification fired exactly once, at the moment a session transitions from ephemeral to full session material (i.e. immediately after successful key derivation), before control returns to Transport.
- A notification fired exactly once during session erasure (§7.3), but *only* if the session had reached full session material at some point – a session that never got past the ephemeral stage produces no such notification, since no real session existed to end.

Both notifications execute synchronously, on whatever task is running the session's APDU handling; they MUST be fast and non-blocking for the same reason the command-dispatch integration point (§8) must be, since both run inline in the middle of the protocol state machine.

9 Session Termination

Upon completion or abort of the NFC session, the Lock MUST securely erase:

- Ephemeral X25519 private key (sk^{eph})
- X25519 shared secret
- HKDF intermediate output (PRK)
- Derived AES-256 session keys (both directions)

Only the long-term Ed25519 identity key persists between sessions. This erase routine SHALL be idempotent and safe to invoke regardless of how far a session progressed (including a session that never advanced past awaiting M1). It is invoked by several trigger conditions defined at the Transport layer (session abort, handshake timeout, link loss, frame-integrity error, signature or auth-tag failure); see Part IV §8.4. The long-term identity key survives every erase and is destroyed only on module teardown.

10 Collaboration with the Application Module

The Session and Application Modules are peers that jointly implement one task – an authenticated, confidential command exchange – and neither half is meaningful alone: a Lock running only this module could authenticate a Phone and open a secure channel to it, but could never act on anything sent over that channel; a Lock running only the Application Module would have commands to dispatch and hardware to drive, but no way to know whether a byte string requesting them came from a trusted device.

This module never calls into the Application Module directly. Every plaintext hand-off in both directions passes through the Communication Module Facade (Part III), which this module sees only as an ordinary, synchronous command-dispatch integration point (§8) – it has no visibility into, and no dependency on, whatever runs on the other side of that boundary. This preserves the peer relationship described here even though a concrete integration component now sits between the two modules in practice.

- **Session → Application:** once M3 is verified, every successfully decrypted and authenticated plaintext byte string is delivered onward unmodified. This module does not parse, validate, or interpret plaintext content.
- **Application → Session:** a plaintext byte string is supplied for encryption with the appropriate directional key and a fresh nonce; this module has no visibility into why that plaintext was chosen.
- **Failure signaling:** an AES-GCM authentication tag mismatch is a failure local to this module and MUST NOT be forwarded onward as a plaintext message – it triggers secure erase and session termination directly (§7).
- This module holds no notion of "commands," "authorization," or "actuation." Those are defined entirely in Part II.

11 Security Properties

Property	Mechanism	Notes
Mutual Device Authentication	Ed25519 signatures over the full transcript	Closes the M1-only binding gap of a two-message design
Confidentiality	AES-256-GCM	Directional keys via HKDF `info` separation
Integrity	AES-GCM authentication tag	16-byte tag per message
Key Agreement	X25519	Ephemeral, per-session
Key Derivation	HKDF-SHA256	Challenges bound into salt
Replay Protection (handshake)	Fresh challenges in signed transcript	Prevents cross-session signature reuse
Forward Secrecy	Fresh X25519 pair per session, destroyed at termination	

Property	Mechanism	Notes
Long-Term Key Protection (Lock)	Software in this revision; hardware secure element planned	See §9.1
Relay Attack Resistance	Not yet implemented	Out of scope this revision; see §9.2

Device authentication (this table) is necessary but not sufficient to authorize an action such as unlocking; that determination belongs entirely to Part II.

12 Explicitly Deferred Items (Session-Layer Scope)

12.1 ATECC608A Secure Element Migration

The Lock currently manages its long-term Ed25519 private key in software. A future revision will migrate long-term key storage and signing to the Microchip ATECC608A secure element, which provides hardware-isolated private key storage, hardware-accelerated signing, and a hardware root of trust for provisioning. **Action item:** confirm the target ATECC608A variant's supported curve set before finalizing whether the long-term identity scheme remains Ed25519 or migrates to ECDSA P-256, as this affects both firmware and the Mobile Application's verification path.

12.2 Relay Attack Resistance

This revision does not defend against relay attacks. Candidate mitigations for a future revision include distance-bounding/RTT measurement layered on top of the handshake timeout T_{HS} (Part IV §8.1), and leveraging NFC's inherent short range as a first-order, non-conclusive mitigation.

Part II
Application Module

1 Scope and Layering

This part specifies the plaintext command set exchanged once a secure session is established, the current authorization policy, and the logic that actuates the physical lock mechanism. This module has no knowledge of cryptographic operations, key material, or transport/APDU framing; it receives only plaintext byte strings that the Session Module has already authenticated as originating from a device holding a provisioned Ed25519 identity, and it returns plaintext byte strings for encryption.

1.1 Purpose

This module owns three responsibilities, and only these three:

1. **Command dispatch** – interpreting the plaintext byte string delivered via the Communication Module Facade (Part III) as a structured command and routing it to the correct handler (§4).
2. **Authorization** – deciding whether an authenticated device is permitted to have a given command executed (§5).
3. **Actuation** – driving the physical lock mechanism and reporting its resulting state (§6), including recovery from mechanical failure (§3).

It MUST NOT implement, reimplement, or depend on the internal details of any cryptographic operation, nonce generation, or key storage. It MUST NOT depend on `session.h`, `transport.h`, `lli.h`, or any PN532 header; its only dependency on the rest of the firmware is the Communication Module Facade (Part III).

1.2 Non-Goals

This part does not cover: cryptographic session establishment or secure messaging (Part I), APDU framing or ISO-DEP timing (Part IV), NFC controller specifics (Part V), or the physical/mechanical design of the lock hardware itself (motor, solenoid, or deadbolt selection). Command sources other than the NFC secure session (a planned BLE out-of-band channel) are deferred; see §9.3.

2 Relationship to Other Components

This module has exactly two external dependencies, of two fundamentally different kinds, and neither is "beneath" it:

- **The Communication Module**, reached exclusively through the Facade (Part III), hands this module authenticated plaintext and accepts plaintext back for encryption. This module has no knowledge of how that plaintext was authenticated or encrypted, and the Communication Module has no knowledge of command content.
- **The lock actuator hardware** (§6) is this module's other, unrelated dependency – physical control that the Communication Module never touches and has no reason to know exists. This is precisely why this module is not simply "the layer that consumes Communication-Module output": roughly half of what it does has nothing to do with communications at all.

A device is authenticated by the Session Module (Part I) before any plaintext ever reaches this module's dispatch logic. This module never sees, and cannot be tricked by, an unauthenticated message – but that guarantee comes from a peer collaborator (via the Facade), not from a layer underneath it.

3 Actuation Model: Asynchronous "Tap-and-Go" Workflow

NFC requires the Phone to remain within close physical proximity (approximately 2 cm) of the Lock. Because mechanical actuation (in particular via a stepper motor) can take several seconds, requiring the user to hold their phone in place until the door physically opens produces poor UX and elevated transaction-failure rates. This specification therefore requires an asynchronous actuation workflow, decoupling the digital transaction from the physical one:

1. **Digital authentication:** the Phone completes the cryptographic handshake (Part I §4) within milliseconds.
2. **Command dispatch:** the Phone sends `CMD_UNLOCK` (or `CMD_LOCK`). This module verifies authorization (§5).
3. **Immediate reply:** this module SHALL queue a success response before physical actuation begins, not after it completes.
4. **Disconnect:** the Phone receives the success response, provides its own UX feedback, and drops the NFC connection.
5. **Physical actuation:** *after* the digital transaction concludes, this module begins driving the actuator.

Requirement: because the digital transaction reports success before the mechanical action completes, this module MUST rely on local hardware, not on the already-closed NFC session, to detect and manage physical failures (§3). This pattern generalizes beyond unlock/lock: any future command whose physical effect genuinely outlasts the response should follow the same reply-first, act-after shape (Part III §8) rather than stretching the Facade's response-wait budget to cover a slow actuator.

4 Mechanical Error Handling

Because the digital transaction reports success before the mechanical action completes, the Lock relies on local hardware to manage physical failures (motor stalls, jams, power loss).

4.1 Local Feedback

Since the user is physically present at the door, the Lock's own hardware SHALL serve as the primary indicator of mechanical outcome, independent of whatever the phone already displayed:

- **Success:** if the mechanical limit switch confirms the target position was reached, the Lock SHALL provide a positive visual and auditory indication.
- **Failure:** if the motor stalls or fails to reach the target limit switch within a defined timeout, the Lock SHALL provide a distinct, unmistakably negative visual and auditory indication, overriding whatever success indication the phone already showed.

4.2 Auto-Reversal for Jams

Requirement: if a stall is detected (via a motor-driver stall-detection signal, or by failing to reach a limit switch within a defined timeout), this module MUST:

1. cease driving the motor in the current direction;
2. immediately reverse the motor back to the fully locked state.

The deadbolt MUST NEVER be left partially engaged, as this is both a security vulnerability and a physical usability hazard.

4.3 Intent Logging (Power-Loss Resilience)

If power is lost during actuation, the Lock boots into an unknown mechanical state. To bound this:

1. **Pre-actuation intent:** before driving the motor, this module SHALL persist a target-state marker to non-volatile storage.
2. **Boot verification:** on boot, this module SHALL read the limit switches. If the mechanical state is intermediate (neither fully locked nor fully unlocked), it SHALL consult the persisted target-state marker and attempt to safely resolve the state – either reverting to locked or completing the interrupted unlock motion.

4.4 State Synchronization and Event History

When the Lock fails mechanically, it MUST report the truth on the *next* connection: on `CMD_GET_STATUS` (§4) or a subsequent actuation attempt, this module SHALL report the true physical state and the most recent mechanical error, allowing the Mobile Application to reconcile its own (already-optimistic) UI state with reality.

5 Command Set

5.1 Plaintext Command Envelope

The plaintext byte string delivered by the Session Module (via the Facade, Part III) is structured as:

Field	Size	Description
OPCODE	1 byte	Identifies the command; see §4.2
ARGS	variable	Opcode-specific arguments, may be zero-length

The response uses the same shape with OPCODE replaced by a status byte (§8). **This OPCODE byte is a distinct namespace from Transport's INS byte** (Part IV §5) – OPCODE lives inside the AES-GCM-decrypted plaintext and is never visible on the wire; the two happening to use similar small hex values is coincidental and MUST NOT be conflated.

5.2 Command Table

Command	OPCODE	Contract
CMD_PROVISION	0x01	**Request:** PROVISION_SECRET (32 bytes) ‖ PHONE_ED25519_PK (32 bytes). **Response (success):** status 0x00 ‖ LOCK_ED25519_PK (32 bytes). **Response (failure):** status 0x01 (APP_STATUS_INVALID_SECRET). See §7 for the full behavioral contract.
CMD_UNLOCK	0x02	**Request:** none. **Response:** status 0x00. Checks authorization (§5), replies immediately, then triggers asynchronous actuation to unlock (§3).
CMD_LOCK	0x03	**Request:** none. **Response:** status 0x00. Checks authorization, replies immediately, then triggers asynchronous actuation to lock.
CMD_GET_STATUS	0x04	**Request:** none. **Response:** status 0x00 ‖ BATTERY_PCT (1 byte) ‖ LOCK_STATE (1 byte) ‖ LAST_ERROR (1 byte). Returns current hardware, bolt, battery, and diagnostic state.
CMD_REVOKE_KEY	0x05	**Request:** TARGET_PHONE_PK (32 bytes). **Response:** status 0x00. Removes the target public key from the persistent authorized-key store.

An OPCODE not listed above MUST be rejected with APP_STATUS_UNKNOWN_CMD and MUST NOT reach the actuation logic in §6. New commands (e.g. for a future OTA opcode, §9.1) are added by extending this table, not by modifying the envelope format.

5.3 Value Enumerations

The status, lock-state, and error codes below are consolidated here as a single normative table; no prior source document defined all of them in one place, so this table is the authoritative reference going forward.

Application Status Byte	Value	Meaning
APP_STATUS_OK	0x00	Command executed (or state read) successfully
APP_STATUS_INVALID_SECRET	0x01	CMD_PROVISION only: the supplied Provision Secret did not match
APP_STATUS_UNKNOWN_CMD	0x02	OPCODE not present in §4.2
APP_STATUS_ACTUATOR_FAULT	0x03	The actuator reported an unresolved/unknown state after an actuation attempt

LOCK_STATE Value	Value	Meaning
LOCK_STATE_LOCKED	0x00	Bolt confirmed locked (limit switch)
LOCK_STATE_UNLOCKED	0x01	Bolt confirmed unlocked (limit switch)
LOCK_STATE_UNKNOWN	0x02	Intermediate or unresolved position (§3.3)

LAST_ERROR Value	Value	Meaning
APP_ERR_NONE	0x00	No error since last clear
APP_ERR_MOTOR_STALL	0x01	Auto-reversal triggered by a detected stall (§3.2)
APP_ERR_ACTUATOR_UNKNOWN	0x02	Actuator failed to resolve to a defined state

Open item: CMD_REVOKE_KEY does not currently define a distinct status for "key not found." Implementations SHOULD treat revocation of a non-existent key as idempotent success (APP_STATUS_OK) unless a future revision defines an explicit not-found code.

6 Authorization Model

This revision's authorization policy is intentionally simple, and is stated explicitly rather than left implicit:

> **Authentication currently implies authorization.** Any device that successfully completes the Session-layer handshake (Part I) is permitted to issue any command in §4.2, including CMD_REVOKE_KEY.

There is no per-device role, scope, or permission check in this revision. Membership in the authorized-key store itself *is* the only authorization gate; CMD_PROVISION and CMD_REVOKE_KEY manage that store's membership but do not introduce differentiated permissions once a key is a member. This is a conscious, reviewed scope limitation, not an oversight; introducing differentiated authorization (an ACL, roles, or time-limited credentials) is deferred; see §9.2.

7 Lock Actuation

7.1 Actuator Abstraction Interface (AAI)

To keep this document independent of the specific motor/solenoid driver circuit, actuation is expressed against a small abstract interface, mirroring the pattern used for Transport's Link Layer Interface (Part IV §2). Any conforming actuator driver MUST implement:

Primitive	Required Behavior
AAI_Engage()	Drives the mechanism to the unlocked position. Returns success/failure once motion completes or a hardware timeout elapses.
AAI_Disengage()	Drives the mechanism to the locked position. Same completion/timeout semantics as AAI_Engage().
AAI_GetState()	Returns LOCKED, UNLOCKED, or UNKNOWN (no position sensor fitted, or sensor fault).

The concrete driver behind the AAI (GPIO assignment, motor driver IC, timing) is out of scope for this document.

7.2 Actuation Sequencing and Safety

1. `CMD_UNLOCK`/`CMD_LOCK` handling MUST perform the §5 authorization check before calling `AAI_Engage()`/`AAI_Disengage()`. The check cannot currently fail for an authenticated device, but the call MUST remain in place so future ACL enforcement (§9.2) has a single insertion point.

2. Immediately before actuation, this module SHOULD confirm (via `comm_module_session_active`, Part III §6) that the session is still active, to avoid actuating on behalf of a session that has already been torn down.

3. `AAI_GetState()` SHOULD be called both before and after actuation so a subsequent status response reflects the mechanism's actual resulting state, not an assumption that the drive command succeeded.

4. This revision does not define retry policy for a mechanism that reports `UNKNOWN` after actuation, beyond returning `APP_STATUS_ACTUATOR_FAULT`; retry policy is a future revision.

8 Provisioning Workflow

8.1 Ownership

This module owns provisioning end-to-end: the physical trigger (e.g. a button press), the Provision Secret, its display (e.g. as a QR code), the comparison of a received secret against it, and the persistent authorized-key store are all Application responsibilities. The Communication Module's role is limited to two Facade-exposed primitives (Part III §6.7) – arming a one-shot handshake exception, and a deferred signature fact-check – and nothing about provisioning otherwise requires this module to depend on anything but the Facade.

8.2 Why Verification Is Deferred, Not Skipped

A design in which handshake signature verification can be switched off is a standing authentication-bypass path, not a provisioning feature merely gated by a button – its risk exists the moment the branch exists, reachable by any future state-machine bug or fault injection on the trigger input, independent of how well the gating itself works. This specification therefore requires that verification is never skipped, only *deferred*: at the moment M3 arrives during a provisioning window, the candidate being registered is by definition not yet in the authorized enumerator (§5, Part I §5), so there is genuinely nothing to check Sig_P against yet – not a reason to skip the check, but a reason to check it moments later, once `CMD_PROVISION` supplies the claimed public key.

8.3 Flow

1. This module detects its own provisioning trigger (e.g. a button press), generates a Provision Secret via a CSPRNG, starts its own provisioning timeout, arms the Facade's provisioning window (Part III §6.7) for that same duration, and renders the secret on its own display peripheral.

2. The Phone scans the displayed secret, taps NFC, and completes an otherwise-ordinary M1/M2/M3 handshake. Because the window is armed, the Session Module (Part I §5) skips resolver-based verification at M3 and instead caches Sig_P and the transcript, unverified, proceeding directly to a full secure session exactly as it would normally – only the M3 candidate-verification step is different, and only because there is nothing yet to verify it against.

3. The Phone sends an ordinary encrypted `CMD_PROVISION` command containing the Provision Secret and its claimed Ed25519 public key, delivered through the normal command path (Part III §4).

4. This module: (a) invokes the Facade's deferred-verification primitive against the claimed key; (b) compares the received secret against its own copy in constant time and confirms its own timeout has not elapsed; (c) only if both succeed, commits the claimed key to its authorized-key store and disarms the provisioning window (single-use); (d) encodes success or failure in the response payload per §8.

Requirement: the provisioning window is single-use. It MUST NOT be automatically re-armed after one session completes, successfully or not; a new attempt requires a new deliberate trigger. A newly provisioned identity becomes usable only on a subsequent connection, participating in the standard authenticated handshake via the ordinary enumerator (Part I §5) – no notification back to the Communication Module is needed, since it never held a copy of the identity store to update.

8.4 Provision Session Restrictions

A provisioning-window session is **not** a general-purpose authenticated session, even though it uses the identical encrypted channel. The phone's long-term identity has not yet been verified at the point the channel opens.

- The Session Module and Communication Module Facade SHALL continue to behave identically to a normal secure session – they decrypt and deliver plaintext without interpreting it, and enforce no provisioning-specific restriction of their own.
- **This module**, not the Communication Module, SHALL enforce that only `CMD_PROVISION` is accepted while its own provisioning state is active.
- If any other command arrives while provisioning is active, this module SHALL reject it, terminate provisioning, and request that the session be torn down (triggering ordinary secure erase, Part I §7) – it MUST NOT silently process an unrelated command mid-provisioning.

Provisioning is complete only once the Provision Secret is validated, the claimed key is supplied, the deferred signature check succeeds, and the key is committed to storage; the provisioning session then terminates normally.

8.5 Deployment Assumptions

The physical trigger for provisioning deserves the same scrutiny as any other trust boundary: it SHOULD be debounced, MUST NOT be remotely triggerable, and SHOULD be logged or otherwise surfaced to the owner when used, since it is the one physical action the entire feature's security rests on. Likewise, if the surface displaying the Provision Secret can be read or photographed by anyone other than the intended installer at the moment of provisioning, the "physical access required" guarantee this feature rests on is void, regardless of anything else in this section. Both are deployment assumptions, not protocol properties, and SHOULD be documented as such wherever this firmware is deployed.

9 Application-Layer Response Semantics

Transport-layer status words (SW1/SW2, Part IV §9) describe only conditions visible to the Transport and Session layers (malformed APDU, handshake failure, auth-tag mismatch); by the time a status word is set, neither layer has visibility into plaintext content. This module therefore defines its own status byte (§4.3), carried as the first byte of the encrypted plaintext response. A Transport- or Session-layer failure never reaches this module and therefore never produces one of these status bytes – those failures are signaled entirely below this module.

10 Collaboration with the Communication Module

This module SHALL:

- Own a single dedicated task that both services the Facade's command mailbox and runs its own recurring work (tamper/integrity monitoring chief among them), using a bounded wait so that integrity/tamper monitoring never depends on NFC activity. See Part III §8 for the full contract this task must honor.
- Encode every application-level outcome inside the response bytes (§8), never via any signal the Communication Module can observe – the Facade's response-completion call has no "this failed" channel, by design.
- Keep its command handler fast, deferring any genuinely slow or physically consequential work to after a prompt reply, per the asynchronous actuation model (§3).
- Treat a session-established notification as advisory only (§9.2 of Part III), never as authorization to act – gate every real action on the specific command received, per §5.
- Depend only on the Facade's public interface (Part III), never on Session, Transport, or LLI directly.

11 Security Properties

Property	Mechanism	Notes
Authorization	Implicit – authentication equals authorization	Any authenticated identity is authorized for any command in §4.2; see §5
Actuation Integrity	Single-writer access to the AAI from the Application's own task	No concurrent-actuation protection beyond the Communication Module's single-session-at-a-time model
Mechanical Safety	Auto-reversal on stall; power-loss intent logging	§3; never leaves the deadbolt partially engaged
Command Replay	Inherited from Session-layer key freshness	This module adds no independent nonce or sequence number
Command Injection	Strict OPCODE allow-list (§4.2)	Unknown opcodes rejected before reaching §6
Provisioning Integrity	Deferred (never skipped) signature verification + single-use window + physical trigger	§7

12 Explicitly Deferred Items

12.1 OTA Firmware Update Dispatch

Planned as new opcode(s) extending §4.2. Open questions: whether OTA payloads route through the existing plaintext budget (§7.2 of Part I) or require Transport-layer chaining (explicitly out of scope there); firmware image authenticity (a Session-layer or provisioning-layer concern, not designed here); and whether OTA requires a stricter authorization check than §5's current all-or-nothing rule, which strongly motivates completing §9.2 first.

12.2 Access Control List (ACL) and Roles

Planned replacement for §5's current policy: an access control list or signed capability per authorized identity; a revocation mechanism beyond `CMD_REVOKE_KEY`'s current flat removal (e.g. short-lived credentials or an online revocation check); and optional role/scope differentiation (owner vs. guest vs. time-limited access), which would extend §6.2's authorization insertion point to be per-command rather than all-or-nothing.

12.3 Out-of-Band BLE Command Source

Planned addition of BLE as a second transport carrying commands into this module, alongside NFC. If reused, this module's contract ("has no knowledge of transport") already accommodates a second transport with no change required here; the work belongs in a future BLE Transport sibling to Part IV. Whether any command should be restricted to one transport only (e.g. disallowing `CMD_UNLOCK` over a longer-range BLE link for relay-risk reasons) is an authorization-policy question and belongs in §9.2.

Part III
Communication Module Facade

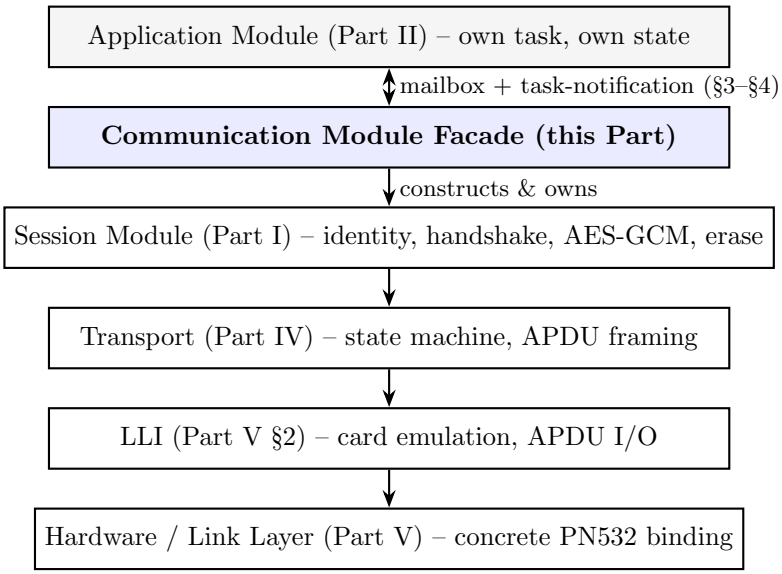
1 Scope and Purpose

This part specifies the Communication Module Facade: the single integration boundary the Application Module (Part II) is permitted to depend on. It wires the Link Layer Interface, Transport, and Session modules into one running unit, owns exactly one task that drives the Transport session loop, and hands commands and session lifecycle events to the Application Module through a bounded, well-defined handoff – so that no component beneath this boundary ever executes application code on its own task.

Requirement (sole dependency boundary): no Application Module source file may depend on the Session, Transport, or LLI interfaces, or any hardware-specific interface, directly. Every capability the Application Module needs from the rest of the firmware MUST be reachable through this Facade. If a capability is missing, the Facade's surface MUST be extended deliberately; the boundary MUST NOT be bypassed.

This firmware targets exactly one PN532 and one lock mechanism, so the Facade is specified as a process-wide singleton rather than a multi-instance handle.

2 Architecture



The boundary between the Application Module and this Facade is a genuine dependency in one direction only (the Application Module depends on this Facade), but is deliberately *not* a synchronous function call across tasks – see §4. Everything from this Facade downward is a conventional dependency stack: this Facade genuinely cannot run without a conforming Session instance beneath it, which genuinely cannot run without a conforming Transport instance, which genuinely cannot run without a conforming LLI implementation (Part IV §2, Part V).

3 Lifecycle Contract

Requirement: the Facade SHALL be driven through the following call order, and SHALL reject or refuse out-of-order calls rather than silently tolerating them:

Call	Contract
Initialize	Constructs the underlying LLI, Transport, and Session instances from a caller-supplied configuration. Does not spawn a task and does not install any interrupt handler of its own – ownership of the single PN532 interrupt line is established as a side effect of initializing the layers beneath this Facade, not by this Facade directly. Safe to discard the configuration after this call returns; the Facade copies it.
Register Application task	MUST be called exactly once, after the Application task exists, before Start. A second call, or a null handle, MUST be rejected. Required because every command and lifecycle event is delivered as a notification targeted at this handle; nothing can be delivered before this call completes.
Start	Spawns the single Facade-owned task and returns immediately. If Register Application task was not called first, this call MUST log an error and MUST NOT spawn a task.
Stop	Requests the Facade-owned task to exit. See §7 for the exact timing guarantee this provides (and does not provide).
Deinitialize	Tears down Session, Transport, and LLI, and clears internal state so that a subsequent Initialize → Register → Start cycle begins from a clean slate. Does not itself stop a still-running task – Stop MUST be called first.

4 Command Handoff Contract

Every plaintext application command decrypted by the Session Module SHALL be delivered to the Application task and SHALL wait for that task's response before the Facade allows Session to encrypt and send an R-APDU. This is a bounded synchronous round-trip, not fire-and-forget, because a Phone is physically waiting over NFC for a reply.

Requirements:

1. The Facade SHALL notify the Application task that a command is waiting, then block internally for up to a configured *application response timeout* awaiting that task's response.
2. If the Application task supplies a response within the timeout, the Facade SHALL deliver it to Session for encryption verbatim.
3. If the Application task does *not* supply a response within the timeout, the Facade SHALL give up waiting, and Session SHALL reply to the Phone with an empty, successful acknowledgement

(per Part I §8's non-OK-handler contract) rather than leaving the phone without any reply. **This is a documented, known behavior, not an oversight:** the Phone will observe a normal-looking success response for a command that was, in fact, never processed by the Application Module. Detecting and mitigating repeated occurrences of this (e.g. a timeout counter feeding a forced-abort, or a system-level watchdog) is an integration-level concern and is not eliminated by this contract.

4. A late response arriving after the timeout has already elapsed MUST be treated as a no-op: it MUST NOT be delivered as the response to a *subsequent* command, and MUST NOT produce any notification that could be mistaken for one.

5. Exactly one command may be outstanding at a time. This follows directly from the underlying NFC protocol being strictly synchronous (one C-APDU outstanding at a time; Part IV §6), so the handoff never needs to queue more than one command or one response.

5 Lifecycle Event Notification Contract

Session establishment and termination (Part I §8) are surfaced to the Application task as **fire-and-forget** notifications, not bounded round-trips like §4 – there is no APDU tied to "a session started," so the Facade does not wait for acknowledgement before letting Transport continue.

Requirements:

1. The Facade SHALL notify the Application task once when a session reaches full authentication, and once when an already-authenticated session ends, using the same notification channel used for command delivery (§4).

2. Because both notification types share one channel and this contract carries only the *latest* pending event, not a queue of events, two lifecycle events that both land before the Application task next wakes SHALL be coalesced to the later one. **This is a known, accepted constraint of a deliberately minimal design**, not an unintentional gap: a session that starts and ends in rapid succession may be observed by the Application task only as "ended." Consumers that require every transition to be individually observable MUST NOT rely on this notification path alone.

3. The Application task MUST check for both a pending event and a pending command on every wake – a single wake may legitimately carry both.

6 API Surface

6.1 Configuration

The Facade's configuration groups parameters that are, in substance, passed through to the layers it constructs (Parts I, IV, V), plus parameters that govern the Facade's own task and timeout behavior:

Group	Contents
Hardware/Link Layer (Part V)	GPIO assignment, I2C port/clock, and card-identity bytes (SENS_RES, NFCID1, SEL_RES, and the optional General/Historical byte fields)
Transport (Part IV)	Handshake timeout (T_{HS}), reader-activation timeout, per-APDU receive timeout
Session (Part I)	The Lock's long-term Ed25519 key pair, and a candidate enumerator satisfying the peer-resolution contract of Part I §5
Facade-owned	The application response timeout (§4); task stack size, priority, and core affinity (each individually defaultable)

All configuration is value-copied at Initialize time; the caller MAY discard its configuration structure immediately after that call returns.

6.2 Types

Type	Values
Facade result code	Success; invalid argument (null pointer, out-of-order call, or destination buffer too small); internal failure (a layer beneath this Facade failed to initialize)
Lifecycle event	None pending; session established; session terminated (§5)

6.3 Function Contracts

Function	Contract
Initialize / Deinitialize	§3
Register Application task	§3
Start / Stop / Force-abort	§3, §7
Query session-active	Returns whether a session currently has full session material (Part I §8), for use by §6.2 of Part II
Poll event	Application-task-only. Returns and clears the pending lifecycle event (§5).
Has command	Application-task-only. Reports whether a command awaits a response, without consuming it.
Get command	Application-task-only. Copies the pending command into a caller-supplied buffer. MUST be rejected if no command is pending or the buffer is too small.
Complete response	Application-task-only. Supplies the response to the pending command in one call, bounds-checked against the plaintext budget (§7.2 of Part I). MUST be a no-op – writing nothing and notifying nothing – if no command is currently pending, so that a late call after §4's timeout cannot corrupt a subsequent command's response.

6.4 Provisioning Primitives

Two functions exist solely to support Part II §7 and expose nothing beyond what that section describes:

Function	Contract
Arm provisioning window	Arms exactly one upcoming session to skip resolver-based M3 verification (Part I §5) and instead cache the M3 signature and transcript for deferred verification, for a caller-supplied duration. Auto-disarms once a session using it completes, successfully or not – it is inherently single-session, not merely single-key; a hard single-key guarantee across multiple attempts within the timeout is the Application Module's responsibility (Part II §7.3), not this Facade's.
Verify provisioned identity	Verifies the cached, unverified M3 signature from the most recently completed provisioning-window session against a caller-supplied candidate public key. A pure cryptographic fact-check: it makes no judgment about secrets, timeouts, or trust – that judgment is entirely the Application Module's, per Part II §7.

No new Session or Transport wire format, and no new Transport INS code, is introduced by provisioning: `CMD_PROVISION` is ordinary application-defined content inside the existing command/response byte arrays (Part II §4), delivered through the existing command-handoff path (§4).

7 What This Facade Deliberately Does Not Add

- No interrupt or GPIO ownership of its own – the single PN532 interrupt is owned entirely by the layer that first needs it, beneath Transport (Part V).
- Exactly one Facade-owned task and exactly one Application task are assumed; this Facade does not spawn worker tasks, pools, or queues on the Application Module's behalf.
- No buffering of more than one in-flight command (§4.5).
- No interpretation of the plaintext OPCODE/ARGS envelope (Part II §4), and no exposure of any internal synchronization structure beyond the accessor functions in §6.3.
- No secret storage, no display/UI ownership, and no persistent identity store, including for provisioning – the Facade's only provisioning-related surface is the pair of primitives in §6.4; the secret, its rendering, and the authorized-key store all remain the Application Module's (Part II §7.1).

8 Known Limitations

- **Stop takes effect only at the next session boundary, and does not block.** The Facade-owned task's loop consults the stop request once per completed Transport session, not once per APDU; a session already in flight runs to completion before the task observes the request and exits. **This call does not block waiting for the task to exit** – a caller that requires a guaranteed-stopped state before Deinitialize MUST confirm task exit by some other means

before calling it. (An earlier internal design note for this Facade described Stop as blocking; that description does not match the current behavior and should be treated as superseded by this requirement, per this document's implementation-is-source-of-truth principle stated in the abstract.)

- **Force-abort cannot interrupt a session already in flight, nor the long wait for a reader to tap.** During the potentially tens-of-seconds-long wait for reader activation, there is currently no cancellation checkpoint; force-abort takes effect only once a reader taps or that wait's own timeout elapses. Closing this gap is scoped future work (a cancel-safe primitive beneath the LLI, Part V), not a guarantee this Facade currently provides. It remains a distinct entry point from Stop specifically so that future work can strengthen it without changing this Facade's public surface.

9 Design Guidance for the Application Task

The Application Module's internals are out of scope for this document (Part II covers only its command/authorization/actuation contract), but the following requirements govern how it MUST use this Facade, since getting this boundary wrong is exactly how a clean separation quietly recouples:

1. **Own a single dedicated task and never block it indefinitely.** That task SHALL both run its own recurring work (tamper/integrity monitoring chief among them) and service this Facade, using a bounded wait – never an unbounded one – so that its own recurring work never depends on NFC activity.
2. Choose the recurring-work period first, and set the application response timeout (§4) comfortably *longer* than it – otherwise that recurring work runs late whenever a command is being processed.
3. **Encode every application-level outcome inside the response bytes**, never via any signal visible to this Facade. A genuinely empty response should be rare and intentional, never a stand-in for "something went wrong."
4. **Keep the response handler itself fast.** This Facade's task is blocked, borrowing time from the underlying NFC protocol budget, for the entire span between reading a command and that handler completing (§4). Genuinely slow or physically consequential work belongs *after* a prompt reply, per Part II §3, not inside the handler.
5. **Treat a session-established event as advisory only.** It means a device proved possession of a registered long-term key – nothing more. Whether that identity may act *right now* is a per-command decision made in the Application Module's own business logic (Part II §5), never inferred from this event alone.
6. **Depend only on this Facade.** No Application source file should depend on Session, Transport, LLI, or any hardware-specific interface; if a needed capability is missing from this Facade, that is a signal to extend it deliberately (§1), not to bypass it.

10 Explicitly Deferred Items

- **A cancel-safe abort primitive beneath the LLI**, so that Force-abort (§7) can interrupt a long reader-activation wait or an in-flight session rather than only taking effect at a natural boundary.

- **Repeated-timeout monitoring.** The command-handoff timeout behavior (§4) is honest but not self-healing; a production deployment SHOULD add monitoring (a counter feeding Force-abort, or a system-level watchdog) for repeated occurrences, which is not something this Facade provides on its own.

Part IV
Transport

1 Scope and Layering

1.1 Purpose

This part specifies the transport that carries the Session Module's (Part I) handshake and secure-messaging bytes over any ISO/IEC 14443-4 (ISO-DEP) compliant contactless link. It is intentionally hardware-agnostic: it defines APDU framing, the instruction set, the target-side protocol state machine, timing requirements, and status codes purely in terms of the ISO/IEC 14443-4 standard and ISO/IEC 7816-4 APDU conventions, with no reference to any specific NFC controller chip. Binding this specification to a concrete controller is the responsibility of Part V, which implements the abstract interface defined in §2 for that controller.

1.2 Component Map

This part's own dependency on Part V is a genuine stack: it cannot function without a conforming implementation beneath it. Its relationship to the Session Module (Part I) is different in kind: it carries Session's bytes as an opaque payload without inspecting or owning them; neither part sits above the other in that relationship. See Part III §2 for how this fits into the complete firmware architecture.

Layer	Responsibility
Session (Part I)	Handshake and secure messaging. Operates on opaque byte payloads; has no knowledge of NFC, APDUs, or RF.
Transport (this Part)	APDU framing, instruction set, protocol-level state machine, ISO-DEP timing, status word dictionary, chaining policy. Depends only on ISO/IEC 14443-4 and ISO/IEC 7816-4 semantics via the LLI.
Hardware / Link Layer (Part V)	Concrete controller driver. Implements the LLI for one specific chip.

A change of NFC controller requires changes only to Part V and its corresponding driver – never to this Part or to Part I.

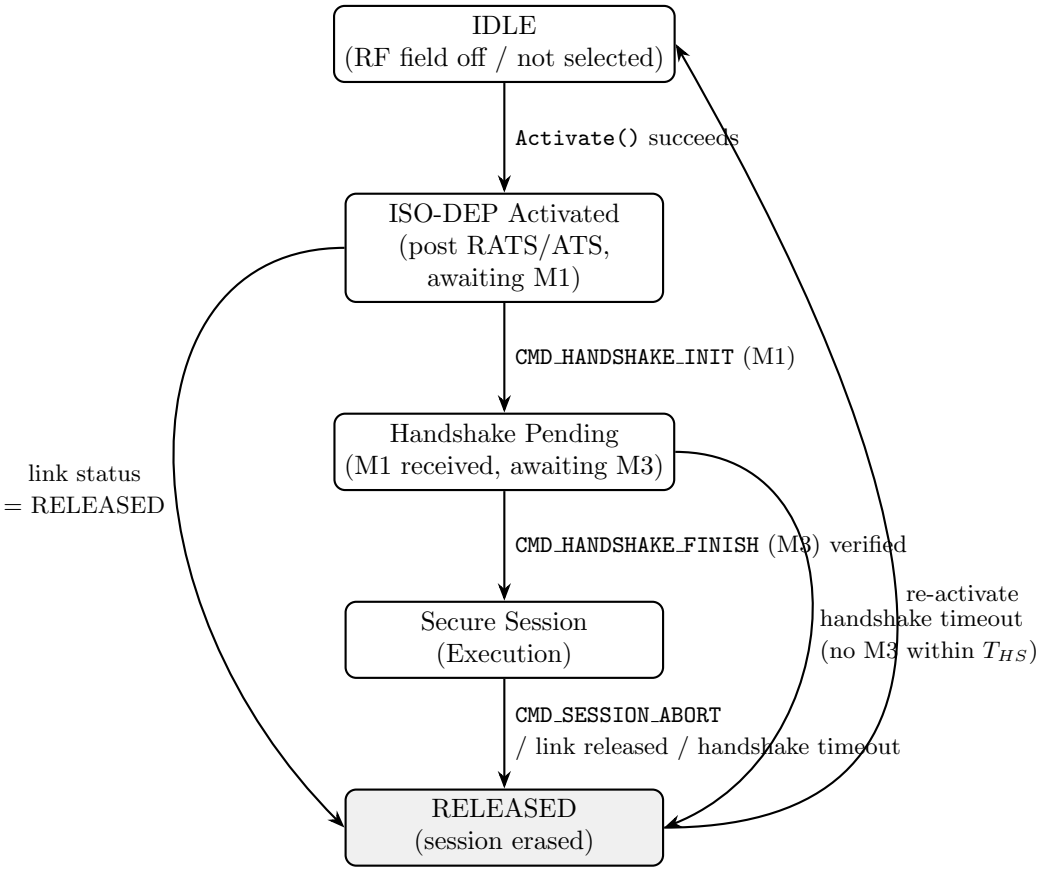
2 Link Layer Interface (LLI)

Any conforming ISO-DEP controller driver MUST implement five primitives. Exact function signatures are a Part V concern; only behavior is normative here.

Primitive	Direction	Required Behavior
Activate	Lock-internal	Performs ISO/IEC 14443-3 polling/anti-collision/selection and ISO/IEC 14443-4 RATS/ATS activation. Returns once the link is in the ISO-DEP Activated protocol state (§3) or reports failure.
Receive APDU (timeout)	Target ← Initiator	Blocks (up to `timeout`) for the next C-APDU. Returns raw APDU bytes, or a defined error (timeout, frame-integrity error, link released).
Send APDU (bytes)	Target → Initiator	Transmits an R-APDU. If the ISO-DEP frame-waiting-time budget is at risk of expiry due to caller processing delay, the driver is responsible for issuing ISO/IEC 14443-4 `S(WTX)` waiting-time-extension blocks per the standard; this is transparent to this Part.
Get link status	Lock-internal	Returns one of: active, released, error. MUST reflect RF field loss or explicit deselection promptly enough to satisfy §8.3.
Abort	Lock-internal	Forces immediate link teardown regardless of current protocol state.

The abstract error taxonomy returned by Receive APDU (timeout / frame-integrity error / link released) is intentionally coarse. A Part V document MUST map its controller's native error/status codes onto this taxonomy; it MUST NOT require this Part to understand controller-specific codes.

3 Target-Side Protocol State Machine



Requirement: `CMD_SESSION_ABORT` (`0x30`) MUST be honored unconditionally in every state from ISO-DEP Activated onward, including ISO-DEP Activated itself (i.e. before M1 has arrived). It MUST NOT be treated as an unrecognized instruction in that state; doing so would leave a session that the reader has explicitly abandoned stuck awaiting M1 instead of transitioning to RELEASED.

4 Data Framing and APDU Encapsulation

Framing follows ISO/IEC 7816-4 Short APDU (Case 4) conventions.

4.1 Command APDU (C-APDU): Phone → Lock

Field	Size	Value / Description
CLA	1 byte	`0x80` (proprietary class)
INS	1 byte	`0x10` / `0x11` / `0x20` / `0x30` (§5)
P1	1 byte	`0x00` (reserved)
P2	1 byte	`0x00` (reserved)
Lc	1 byte	Length of data payload, `0x01`–`0xFF`
Data	*Lc* bytes	Handshake parameters or AES-GCM secure payload

A Short APDU C-APDU is bounded at 261 bytes (CLA+INS+P1+P2+Lc+255 data bytes+Le) per ISO/IEC 7816-4.

4.2 Response APDU (R-APDU): Lock → Phone

Field	Size	Value / Description
Data	0–256 bytes	Handshake response or encrypted application response
SW1	1 byte	`0x90` = normal processing; see §9
SW2	1 byte	Status sub-code

An R-APDU is bounded at 258 bytes.

4.3 Maximum Transmission Unit and Chaining Budget

Handshake payloads (64, 128, and 64 bytes for M1/M2/M3 respectively) fit within a single frame under any ISO-DEP controller.

Flagged reconciliation (see Part I §7.2 for the full discussion): this specification's originally intended plaintext budget for `CMD_SECURE_PAYLOAD` was 227 bytes, computed as $255 - 12 - 16$ against the maximum `Lc`. **The as-built enforcement instead applies the 227-byte threshold to `Lc` itself** – i.e. to the complete nonce ‖ ciphertext ‖ tag package – yielding a true maximum plaintext of $227 - 12 - 16 = 199$ bytes. A `CMD_SECURE_PAYLOAD` C-APDU with `Lc` > 227 MUST be rejected with status word `0x6A 0x80` and MUST NOT be silently truncated or chained; this rejection behavior itself is not in question, only which numeric threshold it should apply. **This document records both figures and defers resolution to a future revision**, per Part I §7.2's disposition. Application-layer chaining remains explicitly out of scope for this revision regardless of which figure is adopted.

5 Instruction Set

Instruction	INS	Description
`CMD_HANDSHAKE_INIT`	`0x10`	Carries M1, 64 bytes
`CMD_HANDSHAKE_FINISH`	`0x11`	Carries M3, 64 bytes
`CMD_SECURE_PAYLOAD`	`0x20`	Carries an AES-256-GCM encrypted application command (Part II)
`CMD_SESSION_ABORT`	`0x30`	Explicit session teardown; triggers immediate key erasure regardless of current state

CLA is `0x80` and P1/P2 are `0x00` for all four instructions. **This INS byte is a distinct namespace from Application's OPCODE byte** (Part II §4.1); INS lives in the unencrypted C-APDU header, while OPCODE lives inside the payload only `CMD_SECURE_PAYLOAD` carries, after decryption.

5.1 State / Instruction Validity

Instruction	Valid State	If Invalid	Effect
`CMD_HANDSHAKE_INIT`	ISO-DEP Activated (awaiting M1) only	`0x69 0x85`	None (no state change)
`CMD_HANDSHAKE_FINISH`	Handshake Pending only	`0x69 0x85`	None if received early; `0x69 0x82` + erase if signature invalid
`CMD_SECURE_PAYLOAD`	Secure Session only	`0x69 0x85`	None (no state change)
`CMD_SESSION_ABORT`	Any state from ISO-DEP Activated onward	n/a (always accepted)	Immediate erase, transition to `RELEASED`

A repeated `CMD_HANDSHAKE_INIT` while already in Handshake Pending or Secure Session MUST be treated as invalid (`0x69 0x85`); restarting a session requires reactivation from IDLE.

6 Phase 1: Handshake and Key Agreement

M1 (Phone → Lock, `CMD_HANDSHAKE_INIT`): Lc `0x40`; 32 bytes pk_P^{eph} ‖ 32 bytes c_P. Receipt transitions the Lock to Handshake Pending and starts the handshake timer T_{HS} (§8.1).

M2 (Lock → Phone): 128 bytes data (32 bytes pk_L^{eph} ‖ 32 bytes c_L ‖ 64 bytes Sig_L), SW1/SW2 = `0x90 0x00`.

M3 (Phone → Lock, `CMD_HANDSHAKE_FINISH`): Lc `0x40`; 64 bytes Sig_P.

Lock response: empty data field. SW1/SW2 = `0x90 0x00` on successful verification (transition to Secure Session), or `0x69 0x82` on failure (transition to RELEASED with immediate erase).

7 Phase 2: Secure Communication

Once in Secure Session, all application payloads use `CMD_SECURE_PAYLOAD`. Phone → Lock: 12-byte nonce ‖ ciphertext ‖ 16-byte tag. Lock → Phone: the same shape under the opposite directional key. An AES-GCM authentication tag mismatch MUST be treated identically to a handshake signature failure.

8 Timing, Liveness, and Session Guardrails

8.1 Handshake Timeout

The Lock MUST enforce a maximum duration T_{HS} between entering Handshake Pending and receiving a verified M3. A recommended starting value is $T_{HS} = 2\,\mathrm{s}$. On expiry, the Lock erases ephemeral state and returns to IDLE exactly as it would on link loss.

8.2 Waiting Time Extension (Protocol-Level)

ISO/IEC 14443-4 defines an `S(WTX)` block allowing a Target to request additional processing time when a response cannot be produced within the negotiated Frame Waiting Time. WTX handling, if needed, is issued transparently within the Send-APDU primitive (§2) – this Part neither issues nor

observes `S(WTX)` blocks directly. Application processing between Receive-APDU and Send-APDU SHOULD remain well under one second wherever possible.

8.3 Link Status Polling

The Lock SHOULD query link status between application-level exchanges – at minimum immediately before committing any state-changing action – and MUST treat a released result identically to an explicit `CMD_SESSION_ABORT`. The maximum acceptable polling interval is a Part V concern.

8.4 Secure-Erase Routine

The following trigger conditions all invoke the identical erase routine defined in Part I §7:

- `CMD_SESSION_ABORT` received in any active state
- Handshake timeout T_{HS} expiry
- Link status reported as released
- A frame-integrity error during Handshake Pending or Secure Session
- Handshake signature verification failure
- AES-GCM authentication tag verification failure

9 Status Words (SW1/SW2) Dictionary

SW1	SW2	Description	Trigger Condition
`0x90`	`0x00`	Success	Command executed correctly
`0x6A`	`0x80`	Incorrect parameters in data field	Invalid X25519 point, malformed Lc, or secure payload exceeding the plaintext budget (§4.3)
`0x69`	`0x82`	Security status not satisfied	Handshake signature verification failed, or AES-GCM auth tag mismatch – both trigger immediate secure erase
`0x69`	`0x85`	Conditions of use not satisfied	Instruction received outside its valid state per §5.1
`0x6A`	`0x81`	Function not supported	Unknown INS byte

10 Conformance Requirements for a Hardware/Link Layer Binding

A Part V document for a specific controller MUST, at minimum: map its activation sequence to Activate (§2), including 106 kbps Type A Passive Card Emulation; map its frame primitives to Receive/Send-APDU, translating controller-native error codes into the timeout / frame-integrity-error / link-released taxonomy; document how it handles `S(WTX)` generation and any bounds on that behavior; specify a concrete link-status polling mechanism and interval; and confirm Short APDU support up to the 261/258-byte bounds assumed in §4, or document a reduced ceiling.

Part V
Hardware / Link Layer – PN532 Binding

1 Scope

This part binds the abstract Link Layer Interface (Part IV §2) to a concrete controller: the NXP PN532 NFC front-end, operating as an ISO/IEC 14443-4 Type A PICC in 106 kbps Passive Card Emulation Mode. It specifies the PN532 host commands and firmware-confirmed behaviors used to implement each LLI primitive, verified against NXP UM0701-02 (PN532 User Manual) and AN133910 (PN532 Application Note). It does not define or alter APDU framing, the instruction set, protocol state machine, timing policy, or status word semantics – those remain normative in Part IV. Should the Lock's NFC front-end ever change, only this Part and its corresponding driver need to be replaced.

1.1 Target Configuration

Parameter	Value
Role	Target (PICC), Card Emulation Mode
Modulation	Type A, 106 kbps, Passive
Protocol	ISO/IEC 14443-4 (ISO-DEP)
Host Interface	I2C (this deployment); SPI/HSU are architecturally possible but not the current binding

2 LLI Primitive Bindings

2.1 Activate → `TgInitAsTarget`

The PN532 is configured for Type A Passive Card Emulation and thereafter autonomously handles `SENS_REQ`, the `SDD_REQ` anti-collision cascade, `SEL_REQ` completion, and `RATS` reception with automatic `ATS` transmission – confirmed firmware behavior per NXP UM0701-02 §4. Activate returns success once the ATS has been sent.

Requirement: the PN532's ISO-DEP configuration (the parameters governing automatic ATS generation and ISO-DEP PICC emulation) MUST be reapplied at the start of every activation, not only once at startup. A silent hardware reset (e.g. following I2C bus recovery) discards this configuration without otherwise making the chip unreachable, which would otherwise produce a Lock that is alive on its host bus but silently unreachable over NFC – a worse failure mode than an outright fault.

2.2 Receive APDU → `TgGetData`

PN532 Status	Meaning	LLI Result
`0x00`	Success	Valid C-APDU returned
`0x01`	Timeout – target has not answered	Timeout
`0x29`	Target released	Link released
any other	CRC/parity/other error	Frame-integrity error (fail closed)

Requirement: on any status other than `0x00`, the driver MUST return an error without passing partial or corrupted data upward. A frame-integrity error during Handshake Pending or Secure Session triggers Part IV's secure-erase routine; a corrupted frame must never reach signature verification or AES-GCM decryption.

2.3 Send APDU → `TgSetData`

A non-success status indicates the R-APDU was not successfully queued; the driver MUST surface this as a send failure rather than assuming delivery.

Waiting Time Extension: confirmed per NXP UM0701-02 §4, the PN532 autonomously issues an `S(WTX)` request if the host does not supply the R-APDU within the negotiated Frame Waiting Time – no additional host logic is required to trigger WTX on this controller. Observed bounds: a default maximum single WTX-extended window on the order of ∼1 second, and Initiator-side stacks commonly cap the number of accepted WTX requests per exchange (a limit this side does not control). Ed25519 signing on the current software implementation completes in low single-digit milliseconds, so WTX is not expected to trigger during the handshake with this controller; a future ATECC608A migration (Part I §9.1) should re-measure its added transaction latency against this budget.

2.4 Get Link Status → `TgGetTargetStatus`

PN532 State	Meaning	LLI Mapping
`PICC_ACTIVATED`	Link up and selected	Active
`PICC_DESELECTED`	Reader paused, has not departed	Active
`PICC_RELEASED` / `IDLE`	Reader departed or link idle	Released
bus/communication error	–	Error

Requirement: a deselect MUST NOT be mapped to released. A deselect/reselect cycle is normal ISO/IEC 14443-4 behavior in which the reader has paused, not departed; treating it as a release would cause Part IV's secure-erase routine to fire during an ordinary protocol pause.

Recommended polling interval: 100–200 ms while in Handshake Pending or Secure Session, and additionally, synchronously, immediately before any state-changing action.

2.5 Abort → `InRelease` / Reactivation

Abort is implemented as a best-effort release sequence that leaves the PN532 ready for the next Activate call to succeed. It is invoked by Part IV's secure-erase routine on session abort, handshake timeout, or any detected integrity failure, so the RF front-end and the cryptographic session state are torn down together.

3 Frame Size Conformance

The PN532's Short APDU support is confirmed per NXP UM0701-02 to satisfy Part IV's bounds without reduction: up to 261 bytes from PCD to PICC, and up to 258 bytes from PICC to PCD. No reduced ceiling applies to either figure discussed in Part IV §4.3's flagged reconciliation. Extended APDU and Type B PICC emulation are not supported by the PN532 and are not used by this system.

4 Command and Status Code Reference

PN532 Command	Code	Response Code
`TgInitAsTarget`	`0x8C`	`0x8D`
`TgGetData`	`0x86`	`0x87`
`TgSetData`	`0x8E`	`0x8F`
`TgGetTargetStatus`	`0x8A`	`0x8B`
`InRelease`	`0x52`	`0x53`

All PN532 host commands are wrapped in the standard TFI/D4 command preamble per the PN532 host protocol; full frame checksums and preambles are defined in NXP UM0701-02 §6 and are not repeated here.

Part VI
Document Set History

1 Provenance and Scope of This Revision

This document is the first revision of the *Firmware Engineering Specification*. It supersedes, for the ESP32 firmware, the prior *Communication Module Specification* (v0.1), and differs from it in four material ways:

1. **Scope widened to the full firmware.** The prior document covered only the Session/Application/Transport/Hardware-Link-Layer stack under the name "Communication Module." This document is scoped explicitly to the entire Lock-side firmware, adding the Communication Module Facade (Part III) as a first-class specified component, since it is now a real, built layer of the system and the Application Module's sole point of contact with everything beneath it.

2. **Application Module content updated to current scope.** The prior document's Application Module part defined only `CMD_UNLOCK`/`CMD_LOCK`/`CMD_STATUS`. This revision reflects the current command set, including `CMD_PROVISION` and `CMD_REVOKE_KEY`, the asynchronous "tap-and-go" actuation workflow, mechanical error handling (auto-reversal, power-loss intent logging, state synchronization), and the provisioning workflow – none of which appeared in the prior document.

3. **Removed implementation-level content.** The prior document's Part V ("Driver Conformance Requirements") recorded eight findings from a code review against the implementation-level document set (struct field types, call-sequence bugs, and similar). All eight findings are reflected as resolved in the current implementation references (`lli.md`, `comm_module.md`); that content is implementation detail and does not belong in an engineering specification, and has been removed rather than carried forward.

4. **One open item carried forward and clarified, not resolved.** The plaintext-budget discrepancy between the originally intended 227-byte figure and the as-built 199-byte figure (Part I §7.2, Part IV §4.3) was not identified in the prior document set at all. It is recorded here as an explicit, flagged reconciliation rather than silently resolved in either direction, pending a decision outside the scope of this document.

A companion *Mobile Application Engineering Specification*, covering the Flutter/phone side of this protocol, is planned as a separate document and will reference this one where the two share a wire contract (the handshake messages of Part I §4 and the command set of Part II §4).

2 Open Items Tracked by This Revision

ID	Item	Status
OI-1	Plaintext-budget reconciliation (199 vs. 227 bytes)	Flagged in Part I §7.2 and Part IV §4.3; not yet resolved in either direction
OI-2	`CMD_REVOKE_KEY` "key not found" status	Flagged in Part II §4.3; interim recommendation given (treat as idempotent success)
OI-3	Cancel-safe abort beneath the LLI	Flagged in Part III §9; scoped future work
OI-4	Repeated command-handoff timeout monitoring	Flagged in Part III §9; an integration-level responsibility, not a Facade guarantee

3 Revision History

Version	Change	Date
0.1	Initial revision. Consolidates and supersedes the Session, Application, Transport, and Hardware/Link Layer parts of the prior *Communication Module Specification* (v0.1); adds the Communication Module Facade as a new, first-class Part; updates the Application Module to its current command set and actuation model; removes the prior document's implementation-level conformance appendix; and records the plaintext-budget discrepancy (OI-1) as an explicit open item rather than resolving it unilaterally.	August 4, 2026